

# Hosting and Deploying ASP.NET Core Application

- *Hosting means providing an environment where an ASP.NET Core application can run and be accessed by users through a network or the Internet.*
- *Deployment is the process of publishing an application and transferring its required files to a server or cloud platform so that users can access it.*

For example, an ASP.NET Core MVC application developed on a student's computer can be published and hosted on IIS. Users can then access it using:

`https://www.example.com`

## Basic Deployment Flow

ASP.NET Core Application



Publish



Hosting Environment



Web Server / App Server



Internet



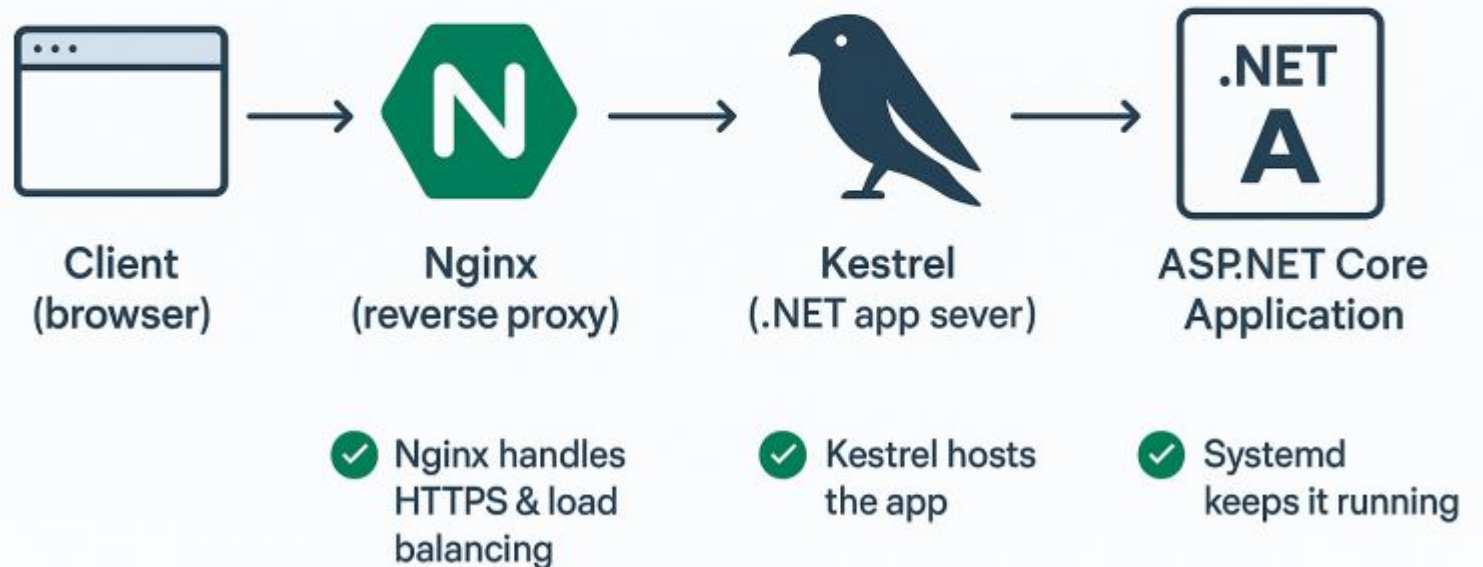
Users

# App Servers and Hosting Methods

ASP.NET Core applications can be hosted using different web servers and hosting models. Common options are:

- *Kestrel*
- *IIS*
- *Nginx*
- *Apache*
- *Docker/Containers*
- *Cloud platforms such as Microsoft Azure*

## Deploying .NET Core App to Linux with Kestrel + Nginx



**Kestrel** is the default, cross-platform web server included with ASP.NET Core.

It can directly receive HTTP/HTTPS requests and execute an ASP.NET Core application.

# Kestre !

## Example

When we run:

```
</> Bash  
  
dotnet run
```

## Advantages of Kestrel

- Cross-platform
- Fast and lightweight
- Included with ASP.NET Core
- Supports Windows, Linux and macOS
- Suitable for development and production

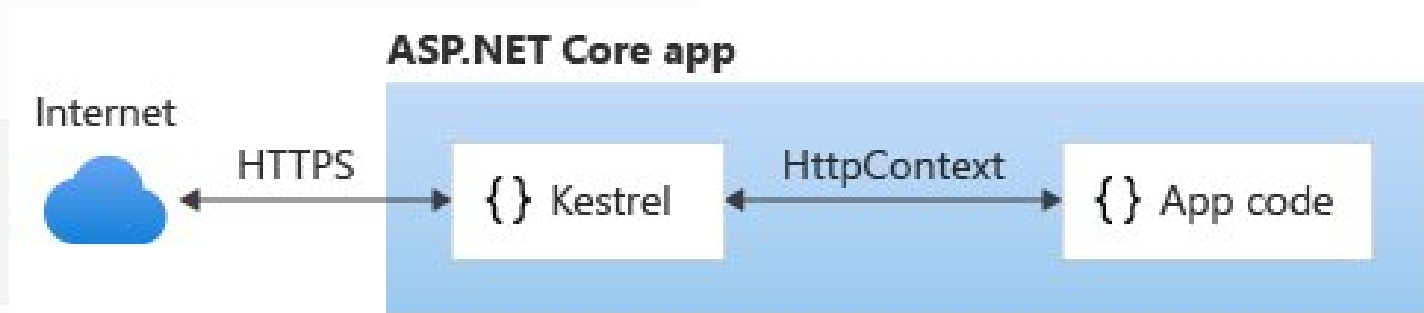
we may see:

```
Now listening on: https://localhost:7042  
Now listening on: http://localhost:5147
```

This means the application is running using **Kestrel**.

The browser can access:

```
https://localhost:7042
```



*However, in many production environments, Kestrel is placed behind another web server such as IIS, Nginx, or Apache.*

# IIS (Internet Information Services)

- *IIS is Microsoft's web server for Windows.*
- *ASP.NET Core applications can be hosted on IIS, where IIS receives client requests and forwards them to the ASP.NET Core application.*

## Example

Suppose an ASP.NET Core application is published to:

```
C:\inetpub\wwwroot\StudentApp
```

IIS can be configured to host this application.

A user can then access:

```
https://college.edu.np/StudentApp
```

## Advantages of IIS

- Windows integration
- HTTPS and certificate management
- Authentication support
- Application pool management
- Logging and monitoring
- Easy integration with Windows-based infrastructure

# ASP.NET Core Module

*The ASP.NET Core Module (ANCM) is an IIS module that helps IIS communicate with ASP.NET Core applications. It acts as a bridge between IIS and the ASP.NET Core application.*

For example:

```
User
↓
IIS
↓
ASP.NET Core Module
↓
ASP.NET Core Application
```

Depending on the hosting configuration, the ASP.NET Core Module can start and manage the application process and forward requests to it.

## Example

If a user requests:

```
https://example.com/student
```

*IIS receives the request. The ASP.NET Core Module helps route the request to the ASP.NET Core application.*

- *Nginx is a popular, high-performance web server and reverse proxy.*
- *It is commonly used to host ASP.NET Core applications on Linux.*

A typical setup is:

```
Client
  ↓
Nginx
  ↓
Kestrel
  ↓
ASP.NET Core
```

Here, Nginx receives requests from clients and forwards them to Kestrel.

## Example

Suppose Kestrel is running on:

```
http://localhost:5000
```

Nginx can receive requests at:

```
https://www.example.com
```

and forward them to:

```
http://localhost:5000
```

Thus, users do not need to directly access the Kestrel port.

## Advantages

- High performance
- Cross-platform
- Reverse proxy support
- Load balancing
- HTTPS support
- Commonly used with Linux servers

# Apache e

- *Apache HTTP Server is another popular web server that can be used with ASP.NET Core.*
- *It can work as a reverse proxy in front of Kestrel.*
- *Reverse Proxy is a server that receives requests from clients and forwards them to the appropriate backend server/ap*

Example:

```
Client → Nginx (Reverse Proxy) → Kestrel → ASP.NET Core App
```

Here, **Nginx acts as the reverse proxy** between the user and the ASP.NET Core application.

## Architecture

```
Client
  ↓
Apache
  ↓
Kestrel
  ↓
ASP.NET Core Application
```

For example:

```
https://www.example.com
  ↓
Apache
  ↓
http://localhost:5000
  ↓
Kestrel
```

Apache is especially useful when an organization already uses Apache for its web infrastructure.

# Docker and Containerization

- **Containerization** is the process of packaging an application together with its required libraries, runtime and configuration into a container.
- **Docker** is a popular platform for creating and running containers.
- Instead of installing the ASP.NET Core application and its dependencies separately on a server, we can package them into a Docker image.

## Traditional Deployment

Server

- ├ .NET Runtime
- ├ Libraries
- ├ ASP.NET Core Application
- └ Configuration

## Docker Deployment

Docker Container

- ├ .NET Runtime
- ├ Libraries
- ├ ASP.NET Core Application
- └ Configuration

The container can then run on different machines that support Docker.

## Simple Dockerfile Example

**</> dockerfile**

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0

WORKDIR /app

COPY ./publish .

ENTRYPOINT ["dotnet", "StudentApp.dll"]
```

*The application can be built into a Docker image and then run as a container.*

### Advantages of Docker

- *Consistent deployment environment*
- *Easy application packaging*
- *Portable across environments*
- *Easy scaling*
- *Useful for microservices*
- *Simplifies deployment and management*

For example:

**</> Bash**

```
docker build -t studentapp .
```

Then:

**</> Bash**

```
docker run -p 8080:8080 studentapp
```

The application can then be accessed through:

```
http://localhost:8080
```

# Publishing an ASP.NET Core Application

For example:

```
</> Bash
```

```
dotnet publish -c Release
```

This creates the files required to run the application.

A typical output directory may contain:

```
publish/  
├─ StudentApp.dll  
├─ StudentApp.deps.json  
├─ StudentApp.runtimeconfig.json  
├─ appsettings.json  
└─ wwwroot/
```

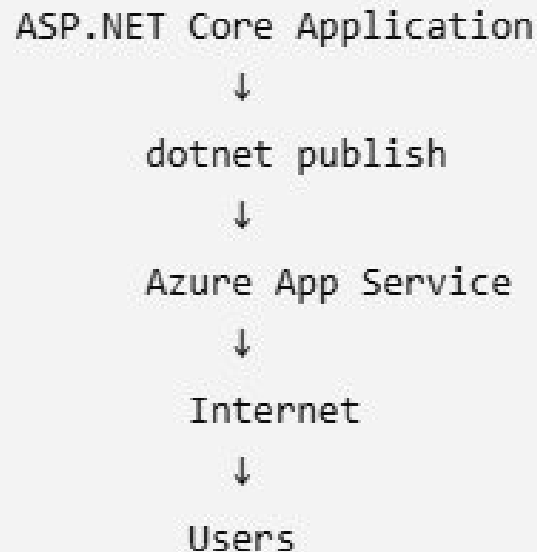
These files can then be copied to a server or included in a Docker image.

*Before deploying an ASP.NET Core application, it is usually published.*

# Publishing to Microsoft Azure Cloud

*Microsoft Azure is a cloud computing platform that can be used to host ASP.NET Core applications. One common option is Azure App Service.*

## Basic Process



## Basic Steps

1. Create an ASP.NET Core application.
2. Test the application locally.
3. Create an **Azure App Service**.
4. Choose the appropriate .NET runtime.
5. Publish the application to Azure.
6. Azure provides a web address for the application.
7. Users can access the application through the Internet.

For example:

ASP.NET Core Application



Publish



Azure App Service



<https://studentapp.azurewebsites.net>

## Advantages of Azure Hosting

- No need to maintain a physical server
- Internet-accessible hosting
- Automatic scaling options
- HTTPS support
- Monitoring and logging
- Integration with other Azure services
- Easy deployment from development tools and source-control systems

## Comparison of Common Hosting Options

| Technology        | Common Environment  | Main Purpose             |
|-------------------|---------------------|--------------------------|
| Kestrel           | Windows/Linux/macOS | ASP.NET Core web server  |
| IIS               | Windows             | Web server and hosting   |
| Nginx             | Linux/Windows       | Web server/reverse proxy |
| Apache            | Linux/Windows       | Web server/reverse proxy |
| Docker            | Windows/Linux       | Containerized deployment |
| Azure App Service | Microsoft Azure     | Cloud hosting            |

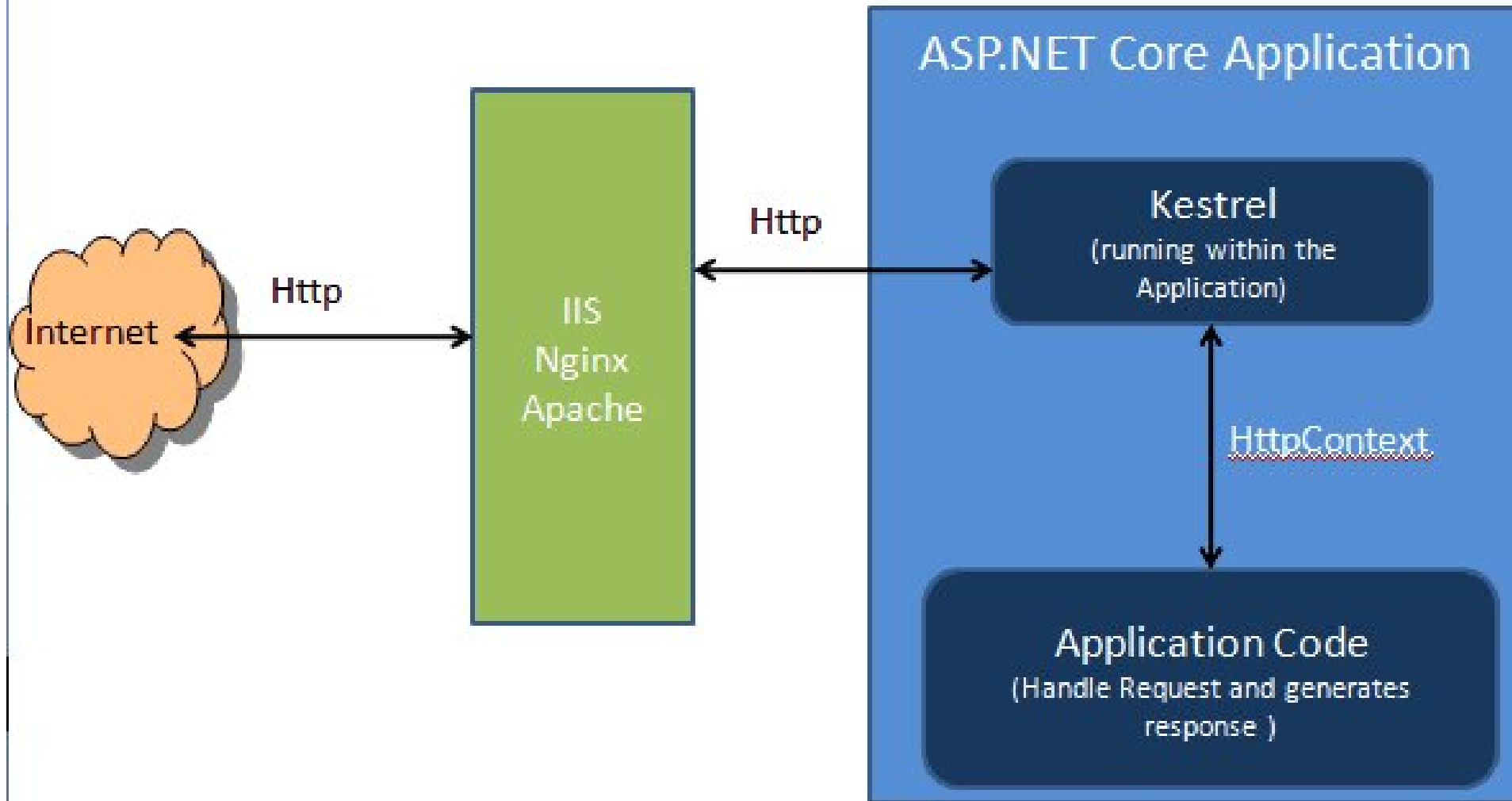
## *Explain about Kestrel Web Server in ASP.NET*

- **Kestrel** is the **built-in, cross-platform web server** for ASP.NET Core applications.
- Introduced with ASP.NET Core to replace the dependency on IIS.
- It is **lightweight, fast, and asynchronous**, built on top of **libuv** (earlier) and now on **.NET sockets**.

### Key Features

1. **Cross-platform** → Runs on Windows, Linux, macOS.
2. **Performance** → High throughput and scalable.
3. **Self-hosting** → Can run independently ( `dotnet run` starts Kestrel ).
4. **Integration** → Can work behind reverse proxies like **IIS, Nginx, Apache** for added security and load balancing.
5. **HTTPS Support** → Handles TLS/SSL certificates.

## Hosting Model of ASP.NET Core Application



## How it Works

- By default, ASP.NET Core apps start with **Kestrel** as the web server.
- Example ( `Program.cs` in .NET 6+):

```
csharp

var builder = WebApplication.CreateBuilder(args);

// Configure Kestrel
builder.WebHost.UseKestrel();

var app = builder.Build();

app.MapGet("/", () => "Hello from Kestrel!");
app.Run();
```

- Runs at `http://localhost:5000` (or `https://localhost:5001`).

### ✅ In short:

*Kestrel is the default, cross-platform, high-performance web server in ASP.NET Core. It can serve requests directly or work with reverse proxies like IIS, Nginx, and Apache in production.*

## Use Cases

- **Development:** Run apps directly with Kestrel.
- **Production:** Usually hosted behind IIS/Nginx/Apache (reverse proxy) for better security and request handling.

# *Discuss about Docker and Containerization*

## **What is Containerization?**

Containerization is a way to package an application and all its dependencies (like libraries, config files, runtime, etc.) into a lightweight, portable unit called a container. This ensures the app runs the same way everywhere—on your laptop, a test server, or in the cloud.

## **Why use Docker with ASP.NET?**

ASP.NET (especially ASP.NET Core) apps are cross-platform, making them ideal for Docker. Containerizing your ASP.NET app:

- Avoids “it works on my machine” issues
- Simplifies deployment
- Enables scaling with tools like Kubernetes

## **What is Docker?**

Docker is the most popular tool for containerization. It lets you:

- Build containers from code
- Run, share, and manage them easily

Think of Docker like a shipping container for software—standardized and ready to go anywhere.

## Step 1: Create a basic ASP.NET Core app

bash

```
1 dotnet new webapi -n MyWebApi
2 cd MyWebApi
```

This creates a simple API with a `WeatherForecast` endpoint.

## Step 2: Add a Dockerfile

Create a file named `Dockerfile` (no extension) in the project root:

dockerfile

```
1 # Use the official .NET SDK image to build the app
2 FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
3 WORKDIR /src
4 COPY . .
5 RUN dotnet publish -c Release -o /app/published
6
7 # Use the lightweight ASP.NET Core runtime image to run the app
8 FROM mcr.microsoft.com/dotnet/aspnet:8.0
9 WORKDIR /app
10 COPY --from=build /app/published .
11 ENTRYPOINT ["dotnet", "MyWebApi.dll"]
```



### Step 3: Build the Docker image

bash

```
1 docker build -t mywebapi .
```

### Step 4: Run the container

bash

```
1 docker run -p 8080:80 mywebapi
```

Now open your browser or use `curl` :

```
1 http://localhost:8080/weatherforecast
```

✓ Your ASP.NET Core app is now running inside a **Docker container**!



### Sample Output (JSON)

json

```
1 v [
2 v   {
3     "date": "2025-11-07",
4     "temperatureC": 12,
5     "temperatureF": 53,
6     "summary": "Cool"
7   },
8 v   {
9     "date": "2025-11-08",
10    "temperatureC": -5,
11    "temperatureF": 23,
12    "summary": "Freezing"
13  },
14 v   {
15    "date": "2025-11-09",
16    "temperatureC": 22,
17    "temperatureF": 71,
```

8

Distinguish between roles and policies. Explain about Kestrel web server in ASP.NET core.

Define claim and policy in terms of authorization. How and when do you create hidden fields?

2. Explain the procedure for building, running and deploying .NET core applications. Write a program to show the possibility of SQL injection attack.[5+5]

4. Define open redirect attacks. Discuss about docker and containerization.[2+3]

10. How can you prevent SQL injection attack? Explain with an example.[5]

7

Explain the chances of SQL injection attack with a scenario.

2

Explain the uses of cookies. What types of vulnerabilities can occur in web application? Show the case of Cross-site Scripting attack.

4

How to you assign roles, claims and policies? Explain with an example.

5

What types of problems lead to SQL injection attack? Discuss any two types of ASP.Net host models.

4

Define open redirect attacks. Discuss about docker and containerization.

Write short notes on (any two):

1. Lambda Expression

2. Containerization using Docker

3. Delegates and Events

12

Write short notes on (any two)

a.LINQ

b. Open Redirected Attack

c. Query String

12